

BUTTONWOOD · SECURITY REVIEW

Buttonwood Cash.

A security review of Cash.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Buttonwood	Cash	Guardian	July 21 – September 28, 2025	October 1, 2025

§ 00

Contents.

119 pages — overview, confidence ranking, scope, 100 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 64 findings	P. 13
	Remediation Review 1 · 35 findings	P. 79
	Remediation Review 2 · 1 finding	P. 117
§ 05	Disclaimer	P. 118
§ 06	About Guardian	P. 119

§ 01

Overview.

Guardian reviewed Cash across 3 review rounds. The final report includes 3 Critical, 9 High, 20 Medium, 66 Low, 2 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	buttonwood-protocol/cash
CLIENT	METHODOLOGY
Buttonwood	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2
PROJECT	MAIN REVIEW COMMIT
Cash	083bc61f665c
PREPARED BY	REMEDATION REVIEW 1 COMMIT
Guardian	5e435899a100
AUDIT WINDOW	REMEDATION REVIEW 2 COMMIT
July 21 – September 28, 2025	f45282bb781f
FINAL REPORT	
October 1, 2025	

1.1 Confidence Ranking

2 / 5

Low Confidence

Guardian assigns a Confidence Ranking of 2 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	3	0	0	0	3
■ High	9	0	1	0	8
■ Medium	20	0	10	0	10
■ Low	66	0	22	0	44
■ Informational	2	0	0	0	2
All severities	100	0	33	0	67

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence ASSIGNED	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Cash in `buttonwood-protocol/cash`. The scope includes 2,524 source lines across 36 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/UsdxQueue.sol	36	75
src/USDx.sol	56	115
src/SubConsol.sol	65	153
src/RebasingERC20.sol	56	107
src/PythPriceOracle.sol	43	108
src/PythInterestRateOracle.sol	48	131
src/OriginationPoolScheduler.sol	228	489
src/OriginationPool.sol	121	231
src/OrderPool.sol	108	203
src/MultiTokenVault.sol	121	253
src/MortgageQueue.sol	111	216
src/MortgageNFT.sol	60	133
src/LoanManager.sol	216	434
src/LenderQueue.sol	81	172
src/GeneralManager.sol	383	745
src/ForfeitedAssetsQueue.sol	37	77
src/ForfeitedAssetsPool.sol	74	157
src/ConversionQueue.sol	155	323
src/Consol.sol	62	124
src/libraries/SharesMath.sol	28	61
src/libraries/Roles.sol	4	11
src/libraries/MortgageMath.sol	276	543
src/libraries/Constants.sol	12	55
src/types/WithdrawalRequest.sol	8	20
src/types/TokenScalars.sol	5	12
src/types/OriginationPoolConfig.sol	14	30
src/types/OPoolConfigId.sol	8	22
src/types/MortgagePosition.sol	25	52
src/types/MortgageNode.sol	8	20
src/types/enums/OriginationPoolPhase.sol	6	14
src/types/enums/MortgageStatus.sol	6	14
src/types/orders/PurchaseOrder.sol	13	29
src/types/orders/OriginationParameters.sol	11	24
src/types/orders/OrderRequests.sol	20	46
src/types/orders/OrderAmounts.sol	6	14
src/types/orders/MortgageParams.sol	13	28

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Expired Orders Do Not Refund Tokens	Logical Error	CRITICAL	RESOLVED	p. 13
C-02	Missing Access Control In Callback	Access Control	CRITICAL	RESOLVED	p. 14
C-03	All USDX Withdrawals Are DoS'ed	DoS	CRITICAL	RESOLVED	p. 15
H-01	Compounding Orders Cannot Be Processed	Logical Error	HIGH	RESOLVED	p. 16
H-02	Mortgage Queue DoS Due To Unused Hint	DoS	HIGH	RESOLVED	p. 18
M-01	MultiTokenVault Inflation Attack	Gaming	MEDIUM	ACKNOWLEDGED	p. 19
M-02	enqueueMortgage Cannot Accept Gas Fees	Unexpected Behavior	MEDIUM	RESOLVED	p. 20
M-03	Incorrect Async Usage	Logical Error	MEDIUM	RESOLVED	p. 21
M-04	ExpandBalanceSheet Allows Lower Rate	Gaming	MEDIUM	RESOLVED	p. 22
M-05	Expand Balance Sheet Has No Restrictions	Validation	MEDIUM	RESOLVED	p. 23
M-06	Large Penalty Payments Arbitraged	Gaming	MEDIUM	ACKNOWLEDGED	p. 24
M-07	Conversion Queue Perturbed	Validation	MEDIUM	RESOLVED	p. 26
M-08	Borrowers Avoid Interest Through Consol	Gaming	MEDIUM	ACKNOWLEDGED	p. 27

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-09	Consol Price Jumps After Queue Actions	Frontrunning	MEDIUM	ACKNOWLEDGED	p. 28
M-10	Re-enqueued Mortgages Trap Gas Fees	Logical Error	MEDIUM	RESOLVED	p. 29
M-11	Incorrect Mortgageld Update Logic	Logical Error	MEDIUM	RESOLVED	p. 30
M-12	Foreclosures Used To Avoid Penalty Payments	Gaming	MEDIUM	ACKNOWLEDGED	p. 31
M-13	First Epoch Of All Origination Pools Can Be DoSed	DoS	MEDIUM	RESOLVED	p. 32
M-14	Collateral Value Not Considered For Foreclosures	Logical Error	MEDIUM	ACKNOWLEDGED	p. 33
M-15	USDx Allows Slippage Free Swaps	MEV	MEDIUM	ACKNOWLEDGED	p. 34
L-01	Forfeited Asset Pool Value Changes Drastically	Logical Error	LOW	ACKNOWLEDGED	p. 35
L-02	Some Lenders Don't Receive Yield	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 36
L-03	Forfeited Assets Queue Zero Value DoS	DoS	LOW	RESOLVED	p. 37
L-04	Cancelled Mortgages Retain The Mortgageld	Logical Error	LOW	RESOLVED	p. 38
L-05	Withdrawal Queue Traps Yield	Logical Error	LOW	ACKNOWLEDGED	p. 39
L-06	Interest Rate Used From Order Creation	Validation	LOW	RESOLVED	p. 40
L-07	Forfeited Asset Pool Value Inflation	Logical Error	LOW	ACKNOWLEDGED	p. 41
L-08	Multiple OPool Origination Discrepancy	Unexpected Behavior	LOW	RESOLVED	p. 42
L-09	Collateral Caps Not Validated On Execution	Validation	LOW	RESOLVED	p. 43

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-10	Expansion Requests Can Unexpectedly Fail	Unexpected Behavior	LOW	RESOLVED	p. 44
L-11	Blacklisted Addresses May Halt Queues	DoS	LOW	ACKNOWLEDGED	p. 45
L-12	Zero Payments Allowed	Validation	LOW	RESOLVED	p. 46
L-13	Refinance Allowed For Completed Mortgages	Validation	LOW	RESOLVED	p. 47
L-14	Conversion Queue May Get Stuck	Warning	LOW	RESOLVED	p. 48
L-15	Converted Positions Can Avoid Penalties	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 49
L-16	Creation Request DoS	DoS	LOW	RESOLVED	p. 50
L-17	Yield Must Be Withdrawable From The Strategy	Warning	LOW	ACKNOWLEDGED	p. 51
L-18	Asset Removal Issues	Warning	LOW	ACKNOWLEDGED	p. 52
L-19	Missing SubConsol Validation	Validation	LOW	RESOLVED	p. 53
L-20	Lacking Configs Validation	Validation	LOW	RESOLVED	p. 54
L-21	Zero Amount Actions Allowed	Validation	LOW	RESOLVED	p. 55
L-22	Lacking Refund Mechanism	Unexpected Behavior	LOW	RESOLVED	p. 56
L-23	Lacking SafeCast Usage	—	LOW	RESOLVED	p. 57
L-24	Convert Rounds In Favor Of The User	Rounding	LOW	RESOLVED	p. 58
L-25	Lacking Expiration Validation	Validation	LOW	RESOLVED	p. 59
L-26	Ineffective Mortgage Request Fills	Unexpected Behavior	LOW	RESOLVED	p. 60
L-27	Actions Can Only Occur During Market Hours	Warning	LOW	RESOLVED	p. 61
L-28	Term And Penalty Payments Can Be Grieved	DoS	LOW	RESOLVED	p. 62

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-29	Check Balance In removeAsset	Unexpected Behavior	LOW	RESOLVED	p. 63
L-30	USDX Withdrawals Can Be DoSed	DoS	LOW	RESOLVED	p. 64
L-31	ForfeitedAssetsQueue Can Be DoSed	DoS	LOW	ACKNOWLEDGED	p. 65
L-32	Incorrect Natspec Comment	Informational	LOW	RESOLVED	p. 66
L-33	Inactive Mortgages In ConversionQueue	Unexpected Behavior	LOW	RESOLVED	p. 67
L-34	Old forfeitedAssetsPool Remains Supported	Logical Error	LOW	RESOLVED	p. 68
L-35	Duplicate Deletion Of Order	Error	LOW	RESOLVED	p. 69
L-36	Early Closures Still Pay Full Interest	Informational	LOW	ACKNOWLEDGED	p. 70
L-37	Race Condition For ForfeitedAssetsPool Tokens	Logical Error	LOW	ACKNOWLEDGED	p. 71
L-38	Penalty Calculation Rounds In Favor Of Borrower	Rounding	LOW	RESOLVED	p. 72
L-39	Initial oPoolAdmin Does Not Have Admin Role	Configuration	LOW	RESOLVED	p. 73
L-40	Penalty Rate Updates Affect Existing Mortgages	Best Practices	LOW	ACKNOWLEDGED	p. 74
L-41	Theft Of Funds From USDX With scalarDenominator	Rounding	LOW	RESOLVED	p. 75
L-42	CEI Pattern Not Followed	Best Practices	LOW	RESOLVED	p. 76
I-01	calculateNewAverageInterestRate Typo Naming		INFORMATIONAL	RESOLVED	p. 77
I-02	Typos	Informational	INFORMATIONAL	RESOLVED	p. 78

REMEDIATION REVIEW 1

H-01	Some Requests Cannot Be Cancelled	Logical Error	HIGH	RESOLVED	p. 79
------	-----------------------------------	---------------	------	----------	-------

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-02	amountForfeited Credits Mortgage Holders	Logical Error	HIGH	RESOLVED	p. 80
H-03	Gas Fees Trapped In enqueueMortgage	Logical Error	HIGH	RESOLVED	p. 82
H-04	Missed Payments Perturbed	Logical Error	HIGH	RESOLVED	p. 83
H-05	Rounding Prevents Action Cancellations	Rounding	HIGH	RESOLVED	p. 85
H-06	Multiple Origination Pools Broken	Logical Error	HIGH	RESOLVED	p. 86
M-01	Mortgage Fee Lost On Expiration	Warning	MEDIUM	RESOLVED	p. 87
M-02	Risk Free Time Difference Arbitrage	Gaming	MEDIUM	ACKNOWLEDGED	p. 88
M-03	Collateral Consumed At Stale Trigger Price	Logical Error	MEDIUM	ACKNOWLEDGED	p. 90
M-04	Converted No Payment Plan Mortgages	Logical Error	MEDIUM	RESOLVED	p. 91
M-05	No Payment Plan Mortgages Arbitraged	Gaming	MEDIUM	ACKNOWLEDGED	p. 92
L-01	Empty Requests Can Be Cancelled	Validation	LOW	RESOLVED	p. 93
L-02	safeTransferFrom Should Come First	Best Practices	LOW	ACKNOWLEDGED	p. 94
L-03	More Than 18 Decimals Incompatible	Validation	LOW	ACKNOWLEDGED	p. 95
L-04	Lacking CEI In Burn	Best Practices	LOW	RESOLVED	p. 96
L-05	Dangerous Withdrawal Request Amounts	Validation	LOW	ACKNOWLEDGED	p. 97
L-06	Lacking Pausability Guards	Validation	LOW	RESOLVED	p. 98
L-07	Missing Reentrancy Guard	Validation	LOW	RESOLVED	p. 99

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-08	Blacklist Prevents Order Processing	Warning	LOW	ACKNOWLEDGED	p. 100
L-09	Lacking Processing CEI	Warning	LOW	RESOLVED	p. 101
L-10	Lacking Length Validation	Validation	LOW	RESOLVED	p. 102
L-11	Outdated Gas Fee Validation	Validation	LOW	ACKNOWLEDGED	p. 103
L-12	Typo	Typo	LOW	RESOLVED	p. 104
L-13	Dangerous Conversion Queue Refund	Best Practices	LOW	RESOLVED	p. 105
L-14	Lacking Individual Borrow Validation	Validation	LOW	ACKNOWLEDGED	p. 106
L-15	Duplicate Origination Pools	Validation	LOW	RESOLVED	p. 107
L-16	Lacking Reentrancy Guards	Validation	LOW	RESOLVED	p. 108
L-17	Unused subConsol Variable	Superfluous Code	LOW	RESOLVED	p. 109
L-18	Incorrect NatSpec Comment	Informational	LOW	RESOLVED	p. 110
L-19	Redundant Check In convert	Superfluous Code	LOW	ACKNOWLEDGED	p. 111
L-20	Redundant Values	Logical Error	LOW	ACKNOWLEDGED	p. 112
L-21	Forfeited Assets Queue Held Up	Warning	LOW	ACKNOWLEDGED	p. 113
L-22	Missing requestWithdrawal Refund	Warning	LOW	ACKNOWLEDGED	p. 114
L-23	Incorrect Event Emission	Logical Error	LOW	RESOLVED	p. 115
L-24	Count Unexpectedly Incremented Twice	Unexpected Behavior	LOW	RESOLVED	p. 116

REMEDIATION REVIEW 2

H-01	Multiple Origination Pools Broken	Logical Error	HIGH	ACKNOWLEDGED	p. 117
------	-----------------------------------	---------------	------	--------------	--------

CRITICAL **C-01**

RESOLVED

Expired Orders Do Not Refund Tokens

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/OrderPool.sol#L163-L176
REVIEW	Main Review

DESCRIPTION

When a user requests a mortgage, either the collateral token or USDX is transferred from the user to the order pool, depending on if the request is compounding or not.

If the order expires before it is processed, the order is simply cancelled and the user's tokens are not returned. The order is deleted.

```
function _processOrder(uint256 index, uint256 hintPrevId) internal returns (uint256
collectedGasFee) {
    // Fetch the order from the orders mapping
    PurchaseOrder memory order = _orders[index];
    // If the order has expired, cancel it. Otherwise, process it.
    if (order.expiration < block.timestamp) {
        // Cancel the mortgage request
        IGeneralManager(generalManager).burnMortgageNFT(order.mortgageParams.tokenId);
        // Delete the order
        delete _orders[index];
        // Emit the PurchaseOrderExpired event
        emit PurchaseOrderExpired(index);
    }
}
```

RECOMMENDATION

Return the user's collateral or USDX depending on the type of order.

PROOF OF CONCEPT <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/pull/3>

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/14>

CRITICAL **C-02**

RESOLVED

Missing Access Control In Callback

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/083bc61f665cd70e71f4e5bae384b88c693e1744/src/GeneralManager.sol#L689>

REVIEW Main Review

DESCRIPTION

The `originationPoolDeployCallback` function lacks access control. A malicious actor can invoke it directly with a `collateralAmount` of 0 and an excessively large `amountBorrowed`. This results in the minting of `amountBorrowed` Consol tokens to the malicious user.

RECOMMENDATION

Implement access control for the `originationPoolDeployCallback` function to prevent unauthorized usage.

PROOF OF CONCEPT <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/pull/4>

RESOLUTION

Status: Resolved.

CRITICAL **C-03**

RESOLVED

All USDX Withdrawals Are DoS'ed

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/083bc61f665cd70e71f4e5bae384b88c693e1744/src/UsdxQueue.sol#L44C1-L50C8
REVIEW	Main Review

DESCRIPTION

Cancelled withdrawal requests are not removed from the queue; instead, the `request.amount` and `request.share` values are set to 0 to allow these requests to be skipped.

However, the `UsdxQueue.processWithdrawalRequests` function does not skip these cancelled requests and [attempts to withdraw](<https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/083bc61f665cd70e71f4e5bae384b88c693e1744/src/UsdxQueue.sol#L45>) zero amounts from the Consol, which reverts with an `AmountTooSmall` error.

Since withdrawal processing is linear, subsequent requests cannot be processed until the previous ones are completed. As a result, a single cancellation causes all subsequent withdrawal requests to be effectively DoSed.

Note that the exact same issue occurs in `ForfeitedAssetsQueue.processWithdrawalRequests` function as well.

RECOMMENDATION

Skip cancelled requests. Do not withdraw from Consol if `request.amount` is zero.

PROOF OF CONCEPT <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/pull/5>

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/15>

HIGH **H-01**

RESOLVED

Compounding Orders Cannot Be Processed

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/083bc61f665cd70e71f4e5bae384b88c693e1744/src/OriginationPool.sol#L205 , https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/083bc61f665cd70e71f4e5bae384b88c693e1744/src/GeneralManager.sol#L689
REVIEW	Main Review

DESCRIPTION

Users create mortgage requests in the `GeneralManager` contract, and these requests are processed by the fulfiller using the `processOrders` function in the `OrderPool`.

During this process, the `GeneralManager.Originate` function is called, which subsequently calls `OriginationPool.deploy`, and finally invokes the `originationPoolDeployCallback` function in `GeneralManager`. The `_enqueueMortgage` function is called within this callback when `originationParameters.conversionQueue` is non-zero.

However, the `_enqueueMortgage` function requires the `mortgageGasFee` to be sent as `msg.value`, but neither the `originate` nor the `originationPoolDeployCallback` functions are payable, making it impossible to send this fee as `msg.value`. As a result, `processOrders` will always fail when a mortgage request has a `conversionQueue` with a non-zero `mortgageGasFee`.

Since compounding orders must include a `conversionQueue`, this issue will have the greatest impact on them. However, it is not limited to compounding orders, as non-compounding orders can also have a `conversionQueue` set.

RECOMMENDATION

Ensure that the required `mortgageGasFee` is passed as `msg.value` during the `processOrders` flow. However, since the fulfiller will invoke this function, the required amount should be transferred from the user to the fulfiller at the time of order creation.

Another important consideration is that the fulfiller can process multiple orders at once and directly using `msg.value` within the `processOrders` loop may introduce new issues. Care should be taken when sending value, and only the required amount for each order should be sent.

PROOF OF CONCEPT <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/pull/2>

H-01 Compounding Orders Cannot Be Processed

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/13>

HIGH **H-02**

RESOLVED

Mortgage Queue DoS Due To Unused Hint

CATEGORY	DoS
LOCATION	MortgageQueue.sol
REVIEW	Main Review

— DESCRIPTION

In the `_insertMortgage` function of the MortgageQueue contract the `hintPrevId` value is only validated and does not functionally serve to reduce the iterations necessary to lodge a mortgage in the right spot in the queue.

When the `triggerPrice` of the head is less than the `triggerPrice` of the newly inserted mortgage the `hintPrevId` always begins from the head. Therefore the insertion must always traverse the full set of nodes to insert a new mortgage.

In many cases the queue will either see an amount of natural usage or an intentional malicious amount of usage such that subsequent mortgages require more iterations than the block gas limit to insert. This would prevent the origination process for compounding mortgages from being fulfilled as well as other processes in the Buttonwood protocol that rely on this underlying queue logic.

— RECOMMENDATION

Do not reset the `hintPrevId` to the head node if the `hintPrevId` has already proven to have a trigger price less than or equal to the `triggerPrice` of the new mortgage.

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/32>

MEDIUM **M-01**

ACKNOWLEDGED

MultiTokenVault Inflation Attack

CATEGORY Gaming
LOCATION MultiTokenVault.sol
REVIEW Main Review

DESCRIPTION

In the `MultiTokenVault` the inflation attack is defended against by using a `decimalsOffset` to preserve a larger amount of shares relative to the number of assets deposited.

However this mechanism can be circumvented because there is a public `forfeit` function which can be used to burn shares of the vault and counteract the `decimalsOffset`, allowing the total initial shares to be manipulated back down to 1 wei by an attacker.

As a result the typical inflation attack can be carried out, resulting in the victim receiving 0 or significantly reduced shares of the vault.

RECOMMENDATION

Firstly, make the `forfeit` function only callable by the `LoanManager` contract.

Even then the inflation attack may still be possible by transferring `Consol` tokens directly to the `LoanManager` before it invokes `forfeit` with it's total balance. Consider either requiring that some initial shares are minted to the dead address, or that `forfeit` cannot be used to forfeit shares below a certain reasonable threshold of `totalShares`.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

enqueueMortgage Cannot Accept Gas Fees

CATEGORY Unexpected Behavior
LOCATION GeneralManager.sol
REVIEW Main Review

DESCRIPTION

In the `GeneralManager` contract the `enqueueMortgage` function is not marked as `payable` and therefore cannot accept the necessary Ether to call the `conversionQueue` with the expected value for gas fees.

As a result the `enqueueMortgage` function cannot be used to enqueue a mortgage as expected.

RECOMMENDATION

Make the external `enqueueMortgage` function on the `GeneralManager` payable so that it can accept the necessary gas fee ether to send to the `conversionQueue`.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/6>

MEDIUM **M-03**

RESOLVED

Incorrect Async Usage

CATEGORY Logical Error
LOCATION LoanManager.sol: 318
REVIEW Main Review

DESCRIPTION

In the `LoanManager` `redeemMortgage` function if the `async` value is provided as true the non-async path will be taken and if the `async` value is false the asynchronous path will be taken.

RECOMMENDATION

Flip the case such that the `async` path is taken if the `async` value is true.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/7>

MEDIUM **M-04**

RESOLVED

ExpandBalanceSheet Allows Lower Rate

CATEGORY Gaming
LOCATION Global
REVIEW Main Review

DESCRIPTION

The `expandBalanceSheet` action does not perform any validation to ensure that the expansion `totalPeriods` matches the `totalPeriods` of the existing mortgage.

As a result, users may create a minimum size expansion directly after initiating a 3-year mortgage at a lower rate, as the three-year treasury rate will typically be lower than the five year treasury rate, and expand their existing mortgage to be extended to 5 years while maintaining this lower rate.

This way those who are given the expansion role are able to game the protocol and obtain a 5-year length mortgage at the rate of a 3-year length mortgage. Even without paying any kind of refinance fee.

RECOMMENDATION

Validate that an expansion can only be made with a `totalPeriods` value that matches the existing mortgage.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/25>

MEDIUM **M-05**

RESOLVED

Expand Balance Sheet Has No Restrictions

CATEGORY Validation
LOCATION GeneralManager.sol
REVIEW Main Review

DESCRIPTION

The `expandBalanceSheet` function in the `GeneralManager` performs no validations that the caller is the owner of the balance sheet being expanded, this allows approved callers of the `expandBalanceSheet` function to extend the length of any user's mortgage at will while expanding their mortgage balance by only the minimum amount.

This results in users holding a mortgage for longer than they expect and requires them to pay more interest than initially promised.

RECOMMENDATION

Perform validation that the caller of the `expandBalanceSheet` function is the owner of the mortgage being expanded

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/26>

MEDIUM **M-06**

ACKNOWLEDGED

Large Penalty Payments Arbitraged

CATEGORY Gaming
LOCATION Global
REVIEW Main Review

— DESCRIPTION

The penalty payments made through the `loanManager` `penaltyPay` function and the interest paid through the `periodPay` function both cause a stepwise increase in the value of consol tokens which can be taken advantage of, especially for large payments.

An opportunistic actor can observe that a large penalty/interest is about to be paid, or have a large outstanding penalty/interest payment they themselves have to make, and frontrun this action to deposit a large amount of USDx into the consol. With a non-trivial share of the total supply of consol shares, the opportunistic actor can realize a significant portion of the penalty/interest payment made and effectively vampire attack this from other Consol depositors who held their shares while the penalty/interest was actually accrued over time.

The opportunistic actor may not be able to instantly withdraw their Consol to realize the immediate stepwise gain, however they profit this amount on paper immediately compared to the other Consol share holders who held for the entire time previous and now had their payment diluted. Furthermore, there may be cases where a large conversion is available through the conversion or forfeited assets queue which does allow the opportunistic actor to enter and exit consol in a very short period while gaining yield which should have accrued to other Consol holders.

— RECOMMENDATION

There are many potential solutions to resolve this stepwise increase in share value issue. A fee for outside deposits into Consol could be added to disincentivize this behavior. The yield paid to the Consol contract could be spread out over time to avoid single stepwise jumps in share value which create such opportunities. Or a deposit delay could be implemented such that there is no guarantee that the actor is able to frontrun the interest/penalty payment value increase.

M-06 Large Penalty Payments Arbitraged

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

RESOLVED

Conversion Queue Perturbed

CATEGORY Validation
LOCATION GeneralManager.sol
REVIEW Main Review

DESCRIPTION

In the `expandBalanceSheet` function there is no validation that requires the caller to provide a nonzero `conversionQueue` address if that mortgage is currently enqueued, even though the mortgage must be re-enqueued into the `ConversionQueue` at the new trigger price.

If this mortgage was previously enqueued in the `ConversionQueue` and is not re-enqueued in the `ConversionQueue` then within the `processWithdrawalRequests` function the mortgage will be executed at the original trigger price when the mortgage was originally enqueued.

This trigger price is not accurate to the trigger price of the new mortgage and in some cases may even reflect a loss for the mortgage's average buy price, if there have not been frequent withdrawals processed, which can result in reverts which block the queue from progressing for an extended period of time.

RECOMMENDATION

If the mortgage being expanded in the `expandBalanceSheet` function is already enqueued in the `ConversionQueue`, require that the conversion queue address provided is nonzero and a valid conversion queue.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/50>

MEDIUM **M-08**

ACKNOWLEDGED

Borrowers Avoid Interest Through Consol

CATEGORY Gaming
LOCATION Global
REVIEW Main Review

— DESCRIPTION

When a large percentage of Consol holders have pending withdrawal requests in the various queues a large mortgage holder can recoup a large portion of their interest payments by simply depositing USDx into Consol and holding consol while they make their payments. The borrower can then process the queue entries as they are available so that each entry returns their share of the yield to the borrower's Consol holdings.

In the case where a large mortgage exists and a significant portion of Consol holders are deposited in a queue with entries that can be processed or can be processed soon the mortgage owner can simply pay off their entire mortgage and then recoup the interest in a short period.

— RECOMMENDATION

To disincentivize this behavior a fee could be levied directly to deposits made to Consol through USDx and outside of the Buttonwood system. Otherwise, there can be caps on the percentage of Consol that is allowed to be deposited into queues at any given time to limit the feasibility of this scenario.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

Consol Price Jumps After Queue Actions

CATEGORY Frontrunning
LOCATION Global
REVIEW Main Review

— DESCRIPTION

Throughout the codebase there are several invocations to `burnExcessShares` throughout the various withdrawal queues. This function call creates a stepwise jump that re-distributes the yield from the holder who is withdrawing their tokens from the queue to the rest of the Consol holders.

This can cause a significant stepwise jump in the Consol token price, especially for large Consol withdrawals which have spent a non-trivial amount of time in the queue. This is likely given the rate of interest payments made and length of mortgages in the Buttonwood cash protocol.

This stepwise jump can be arbitrated by other users who obtain consol right before this stepwise jump and lock in a guaranteed profit that otherwise they would have had to wait to accumulate honestly.

— RECOMMENDATION

Instead of retro-actively burning the excess shares of the users who deposit into the withdrawal queue, consider removing them from the Consol token pool by withdrawing their assets from the Consol vault immediately upon their request to withdraw in the queue.

If the request is cancelled, then simply deposit these assets back into the Consol token at the updated share price.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-10**

RESOLVED

Re-enqueued Mortgages Trap Gas Fees

CATEGORY Logical Error
LOCATION ConversionQueue.sol
REVIEW Main Review

DESCRIPTION

When dequeuing and re-enqueuing a mortgage a new gas fee is required to be covered by the user, however the gas fee provided for the original queue position is not recovered. As a result, this gas fee amount is left trapped in the `ConversionQueue` contract.

RECOMMENDATION

Refund the gas fee collected by the old, queued mortgage and validate that the current gas fee has been provided.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/37>

MEDIUM **M-11**

RESOLVED

Incorrect MortgageId Update Logic

CATEGORY Logical Error
LOCATION MortgageNFT.sol
REVIEW Main Review

DESCRIPTION

The `updateMortgageId` function is flawed because it does not remove the association of the previous mortgageId with the tokenId. Therefore, the tokenId is still associated with the old mortgageId in the `getTokenId` mapping.

As a result, one tokenId could be used to occupy many common mortgageId's, or these leftover `getTokenId` mapping values can cause confusion and issues for consumers of this mapping in normal use.

RECOMMENDATION

Be sure to remove the `getTokenId` entry for the previous mortgageId in the `updateMortgageId` function.

RESOLUTION

Status: Resolved.

MEDIUM **M-12**

ACKNOWLEDGED

Forclosures Used To Avoid Penalty Payments

CATEGORY Gaming
LOCATION Global
REVIEW Main Review

— DESCRIPTION

When foreclosing a position into the forfeited assets queue the amount of forfeited asset tokens minted corresponds to the outstanding debt of the position without interest. It is one issue that this amount does not include interest.

However, this amount also does not include any penalty payments that the mortgage holder had outstanding. This makes sense from the third party Consol holder's perspective. However, from the perspective of the mortgage holder, if they have accrued unpaid penalty payments up to this point then it makes sense for them to instead obtain Consol, enter the forfeited assets queue and allow their mortgage to foreclose to redeem their underlying collateral without the penalty payments.

— RECOMMENDATION

To protect against this potential gaming, consider setting aside the unpaid penalties from the collateral seized on foreclosure and distributing it to all Consol holders similar to how would be done in the `penaltyPay` function.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-13**

RESOLVED

First Epoch Of All OriginationPools Can Be DoSed

CATEGORY DoS

LOCATION <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/OriginationPoolScheduler.sol#L436C1-L445C74>

REVIEW Main Review

DESCRIPTION

The `deployOriginationPool` function does not have access control, and pools are deployed based on predetermined configs added by the admin.

However, the function does not check if the config for the user-provided `oPoolConfigId` exists, allowing pools to be deployed with an empty config.

A malicious actor can front-run the admin just before a new config is added, fetch the `oPoolConfigId`, and deploy a pool before the config is stored. The resulting pool is unusable, as the `Consol` and `USDx` addresses are set to 0. Since `OriginationPool` deployments can only occur once per epoch, the initial epoch is effectively DoS'ed by this action, and the correct pool with a legitimate config can only be deployed in the next epoch.

RECOMMENDATION

If the function is intended to be external, add a check to ensure that `config.consol != address(0)` and verify that a valid config exists for the provided `oPoolConfigId`.

Additionally, consider restricting the `deployOriginationPool` function to admin access only.

PROOF OF CONCEPT <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/pull/1>

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/9>

MEDIUM **M-14**

ACKNOWLEDGED

Collateral Value Not Considered For Foreclosures

CATEGORY Logical Error
LOCATION Global
REVIEW Main Review

— DESCRIPTION

Foreclosures only occur if the mortgage owner misses more than `MAXIMUM_MISSED_PAYMENTS` payments. There is no check on the backing collateral value of the mortgage. When the market declines and the collateral value drop significantly, there is no incentive to continue making monthly payments to keep the collateral and redeem it later.

— RECOMMENDATION

Consider defining a safe ratio between the backing collateral value and the debt and allow mortgages to be foreclosed if the collateral value falls below that ratio, even if there are no missed payments. Additionally, allow borrowers to add more collateral to secure their loans.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-15**

ACKNOWLEDGED

USDX Allows Slippage Free Swaps

CATEGORY MEV
LOCATION USDX.sol
REVIEW Main Review

— DESCRIPTION

The USDX contract is a `MultiTokenVault` with no overrides on the deposit or withdrawal functions, and the conversion functions simply adjust the token amount by the relevant decimals. Therefore, asset A can be deposited and asset B can be withdrawn at exactly the same `exchangeRate`.

This allows for a slippage free swap on the supported tokens, e.g. USDC and USDT. If one token depegs by even a small amount USDX would be immediately arbitrated to hold the depegged token at the cost of all USDX holders.

— RECOMMENDATION

This could be addressed in many ways, including but not limited to adding an exchange rate mechanism for the tokens being deposited and withdrawn from USDX, adding a withdrawal fee that applies when withdrawals are made shortly after deposits, or requiring a cooldown period for withdrawals.

Otherwise acknowledge and be aware of the risk of arbitrage in the case where one token backing USDX depegs, or a user simply wishes to abuse a slippage free swap.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

ForfeitedAssetPool Value Changes Drastically

CATEGORY Logical Error
LOCATION ForfeitedAssetsPool.sol: 109-127
REVIEW Main Review

DESCRIPTION

When a mortgage is foreclosed, the entire remaining collateral is deposited into the forfeited assets pool, and an `amountOutstanding` amount of `ForfeitedAssetPool` tokens is minted. Neither the mint amount nor the added collateral value is relevant to the current state of the forfeited assets pool. As a result, each foreclosure can significantly change the value of a single `ForfeitedAssetsPool` token.

Users who want to withdraw from the `ForfeitedAssetsPool` burn their `Consol` tokens and enter the queue, expecting to get underlying assets from the pool based on the current value. However, there is no guarantee that the same amount of `ForfeitedAssetsPool` token will be worth a similar value when this withdrawal request is processed, and users may end up withdrawing their `Consol` tokens at a much lower value than expected.

RECOMMENDATION

Consider allowing users to provide a minimum value when requesting a withdrawal from the `ForfeitedAssetsPool`, or clearly document this behavior, since users who enter the `ForfeitedAssetsQueue` should be aware of it.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Some Lenders Don't Receive Yield

CATEGORY Unexpected Behavior
LOCATION UsdxQueue.sol
REVIEW Main Review

— DESCRIPTION

Due to the structure of the USDX queue, some lenders who deposited into the origination pool and received Consol tokens will not be able to withdraw their principal or yield for a significant period of time.

Because a queue was chosen for this process, the lenders who submit their withdrawal requests in the USDX queue first will be fully redeemed, with interest, entirely first. And the ones who queue after them will have to wait until the final mortgages are fully repaid to redeem their principle and realize any interest.

As a result, some lenders will be able to withdraw within hours to days, while others will have to wait months to withdraw any amount of their Consol backing amount.

— RECOMMENDATION

Consider restructuring the Consol redemption such that all Consol holders can withdraw a portion of their backing tokens as principal and interest is paid down over time, where this portion increases for all Consol holders evenly over time as payments are made.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

RESOLVED

Forfeited Assets Queue Zero Value DoS

CATEGORY	DoS
LOCATION	ForfeitedAssetsPool.sol
REVIEW	Main Review

DESCRIPTION

The forfeited assets queue burn function iterates through all of the supported tokens and transfers them out to the burner relative to their holdings of the forfeited assets shares.

However, if any of the redeemedAssets tokens reverts on zero value transfers and the corresponding contract balance is empty, due to no forclosures occurring for that token, then a zero-value transfer revert will occur.

This will result in a stuck forfeited assets queue until a forclosure occurs for that asset or the admin sends some tokens to the contract to correct it.

RECOMMENDATION

Only perform the `safeTransfer` call if the amount to transfer is nonzero.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/34>

LOW **L-04**

RESOLVED

Cancelled Mortgages Retain The MortgageId

CATEGORY Logical Error
LOCATION MortgageNFT.sol
REVIEW Main Review

DESCRIPTION

In the `burn` function of the `MortgageNFT` contract there is no clearing of the `getMortgageId` or `getTokenId` mappings for the relevant `tokenId`. Therefore, the `tokenId` continues to occupy the entries in these mappings and act as if it still existed.

RECOMMENDATION

Remove the entries associated with the `tokenId` that is being burned in the `burn` function.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/33>

LOW **L-05**

ACKNOWLEDGED

Withdrawal Queue Traps Yield

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

For the last holder of the Consol token the yield generated is trapped in the withdrawal queue contract and cannot be recovered by the protocol.

Consider the following series of actions using a withdrawal request and subsequent cancellation as an example:

- 100 total shares 100 total tokens
- Request withdrawal of 100 shares worth 100 tokens
- Withdrawal time: 100 shares worth 110 tokens, price per share = 1.1
- 100 total shares 110 total tokens
- $\text{SharesMinted} = 100 * (100 - 100) / (110 - 100) > (100 - 100 \text{ totalShares is } 0) > 100$
- $\text{Shares} - \text{shares minted} = 0$
- No Adjustment for totalSupply or the sender's balance
- $\text{Transfer}(100)$
- $\text{Convert to shares}(100) > \text{shares} = 100 * 100 / 110 = 90.909 \text{ shares transferred}$
- $100 - 90.909 = 9.091 \text{ shares representing } 10 \text{ tokens remain trapped in the withdrawal queue contract.}$

If the final Consol divestment is large this could result in a significant amount of funds trapped in the lending queue.

RECOMMENDATION

Consider adding a way to rescue any yield that could be trapped in the lending queue that would occur when the last Consol holder divests or cancels from the queue.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

RESOLVED

Interest Rate Used From Order Creation

CATEGORY	Validation
LOCATION	GeneralManager.sol
REVIEW	Main Review

DESCRIPTION

In the `_prepareOrder` function the interest rate for an order is fetched from the `PythInterestRateOracle`. However, the interest rate recorded from the oracle at the time of request creation may be significantly different than the current rate in effect when the mortgage is actually originated.

Since there is no validation that the expiry is at most a certain amount in the future it's possible that some mortgage requests could stick around in the `OrderPool` and use stale interest rates when originated.

RECOMMENDATION

Consider validating that the expiry is not too far in the future upon creation requests.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/35>

LOW **L-07**

ACKNOWLEDGED

Forfeited Asset Pool Value Inflation

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

When assets are forfeited to the forfeited assets pool during foreclosure the `amount` of outstanding debt is minted as forfeited asset shares, however the entire mortgage collateral is deposited into the pool.

In cases where the mortgage position has already had payments made these changes the exchange rate of forfeited asset shares and allows the Consol holder at the front of the forfeited assets queue to withdraw more value than their consol should be worth.

For the most explicative example, consider a mortgage where all but 1 wei of the `termBalance` has been paid off, and the collateral is 10 BTC. The amount flash swapped from the Consol contract is simply 1 wei, and therefore the amount of new forfeited assets shares minted is just 1 wei. However, 10 BTC goes into the forfeited assets pool, and all of a sudden, the first Consol holder in the queue can redeem 1 wei of Consol for 10 BTC, while the queued redemptions behind them do not get the liquidity they should have to exit from this foreclosure.

— RECOMMENDATION

Either mint and transfer the current dollar amount of all collateral forfeited to the Consol contract, thus increasing the balance of all Consol holders and allowing them to exit fairly through the forfeited assets queue or consider converting between the amount of forfeited assets shares and underlying dollar value of collaterals in the forfeited assets pool so that Consol holders can be redeemed at a 1 USD value per Consol like the other exit queues.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

RESOLVED

Multiple OPool Origination Discrepancy

CATEGORY	Unexpected Behavior
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The whitepaper suggests that multiple `OriginationPool`'s can be used to originate a loan for an order, however the order fulfillment flow in the `OrderPool` contract only allows for one `OriginationPool` that has been specified by the user to be used with the mortgage origination.

This may limit the size of individual mortgages that can be created with a single request, and deviates from the whitepaper.

RECOMMENDATION

Confirm if this is the expected behavior or not, and if the whitepaper is simply outdated.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/46>

LOW **L-09**

RESOLVED

Collateral Caps Not Validated On Execution

CATEGORY Validation
LOCATION GeneralManager.sol
REVIEW Main Review

DESCRIPTION

The minimum and maximum caps for the chosen collateral token are validated upon a mortgage request but not upon the execution of that mortgage request.

Therefore, if these caps are updated in between the request and execution the resulting order execution may violate the most up to date caps.

RECOMMENDATION

Consider validating the minimum and maximum caps for an order during the execution of that order in addition to the request creation.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/36>

LOW **L-10**

RESOLVED

Expansion Requests Can Unexpectedly Fail

CATEGORY Unexpected Behavior
LOCATION ConversionQueue.sol
REVIEW Main Review

DESCRIPTION

In the origination flow, if the order being executed is an expansion order the `reenqueue` boolean is always marked as true when a conversion queue is provided.

However, if the mortgage being expanded is not already in the conversion queue, then this enqueue request will fail with the `TokenIdNotInQueue` error.

This leads to unexpectedly un-executable expansion orders which must be re-requested without a conversion queue and enqueued in a separate transaction.

RECOMMENDATION

Consider only re-enqueuing a mortgage if that mortgage is already present in the `ConversionQueue`.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/50>

LOW **L-11**

ACKNOWLEDGED

Blacklisted Addresses May Halt Queues

CATEGORY DoS
LOCATION Global
REVIEW Main Review

— DESCRIPTION

The conversion queue transfers collateral tokens directly to the withdrawer's address. As a result, if the withdrawer is blacklisted for the underlying collateral token, they can halt the withdrawal queue due to a revert which occurs when attempting to transfer to their blacklisted address.

A similar action can occur that prevents the Forfeited assets queue from functioning.

— RECOMMENDATION

If the transfer fails in any queue, consider incrementing a mapping value whereby the withdrawer can claim the collateral token in a separate transaction. Otherwise, be sure to only list collateral tokens which do not have a blacklist mechanism.

— RESOLUTION

Status: Acknowledged.

LOW **L-12**

RESOLVED

Zero Payments Allowed

CATEGORY	Validation
LOCATION	LoanManager.sol
REVIEW	Main Review

DESCRIPTION

In the `periodPay` and `penaltyPay` functions there is no validation that prevents a 0-amount payment from occurring. This could lead to unexpected issues since a zero payment is not an expected operation. Furthermore, a minimum amount validation could be performed to add additional safety checks.

RECOMMENDATION

Validate that the amount being paid in the `periodPay` and `penaltyPay` functions is nonzero and ideally introduce a minimum amount validation for these payments.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/30>

LOW **L-13**

RESOLVED

Refinance Allowed For Completed Mortgages

CATEGORY	Validation
LOCATION	LoanManager.sol
REVIEW	Main Review

DESCRIPTION

The `refinanceMortgage` function performs no validation that prevents a user from re-financing a mortgage that is nearly entirely paid off or entirely paid off already. This allows the user to create a mortgage with a term balance of 0 which can lead to unexpected edge cases in the protocol.

RECOMMENDATION

Validate that the mortgage being refinanced has above a minimum threshold of debt before allowing the refinance to occur.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/31>

LOW **L-14**

RESOLVED

Conversion Queue May Get Stuck

CATEGORY Warning
LOCATION ConversionQueue.sol
REVIEW Main Review

DESCRIPTION

In the `processWithdrawalRequests` function for the `conversionQueue`, the `numberOfRequests` is only decremented when a withdrawal request is fully fulfilled. Additionally, there is no maximum limit on the size of a withdrawal request.

Therefore, for large withdrawal requests it is possible that the queue processing can be halted due to the invocation of the `processWithdrawalRequests` function with even a 0 `numberOfRequests` value. This is possible if there are many mortgages near the minimum size which can be matched against a single large withdrawal request, requiring that many loop iterations be completed before the `processWithdrawalRequests` function can finish execution, which could use more than the block gas limit to fully process.

RECOMMENDATION

To resolve this issue, consider implementing a maximum size for the individual withdrawal request amount, such that it is a multiple of the minimum mortgage size which can be processed in a single block.

Otherwise, adapt the usage of the `numberOfRequests` value in the `processWithdrawalRequests` function such that it decrements upon every loop iteration, even when the withdrawal request is only partially processed and a mortgage is fully processed.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/47>

LOW **L-15**

ACKNOWLEDGED

Converted Positions Can Avoid Penalties

CATEGORY Unexpected Behavior
LOCATION Global
REVIEW Main Review

— DESCRIPTION

When a position is fully converted without making any period payments the `termBalance` becomes 0. As a result, the position no longer accrues late payments in the `_applyPendingMissedPayments` function and cannot be foreclosed on as a result.

For any position which had accrued penalty payments and then gets entirely converted they will not have any time constraint on paying these penalties as foreclosure cannot occur.

— RECOMMENDATION

The user still has to pay off these penalties to redeem their position, and this serves as the only incentive to do so. Be aware of this case where a mortgage cannot be foreclosed on and penalty payments can be put off.

— RESOLUTION

Status: Acknowledged.

LOW **L-16**

RESOLVED

Creation Request DoS

CATEGORY	DoS
LOCATION	GeneralManager.sol
REVIEW	Main Review

DESCRIPTION

The `requestMortgageCreation` function accepts a `creationRequest` object with a `mortgageId` NFT id attribute to mint the corresponding mortgage NFT id to the caller.

However, if this `mortgageId` is already taken then the `mint` invocation and thus the higher level `requestMortgageCreation` function invocation reverts.

As a result, it may be possible for a malicious actor to frontrun and DoS a user's calls to the `requestMortgageCreation` function by taking their specified `mortgageId` first.

RECOMMENDATION

Be aware of this and consider refactoring the mortgageId logic to be keyed based on the owner of the mortgage.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/17>

LOW **L-17**

ACKNOWLEDGED

Yield Must Be Withdrawable From The Strategy

CATEGORY Warning
LOCATION SubConsol.sol
REVIEW Main Review

— DESCRIPTION

In the `SubConsol` contract there is no way to withdraw yield from the underlying `yieldStrategy` beyond the collateral deposited into it. This may be unexpected depending on the intended use-case for the `yieldStrategy` and the target `yieldStrategy` implementations.

— RECOMMENDATION

Be sure there is a way to withdraw the yield from the `yieldStrategy` so that this amount is not lost.

— RESOLUTION

Status: Acknowledged.

LOW **L-18**

ACKNOWLEDGED

Asset Removal Issues

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

In the case where an asset is being updated as no longer supported and has a nonzero balance in either the USDx or Consol contract a stepwise decrease in the vault shares value will occur.

In the case of Consol this creates a DoS for the queues which rely on the share price increasing monotonically due to the `burnExcessShares` function.

— RECOMMENDATION

Be aware of this risk and be sure to never delist an asset that has a nonzero balance in the Consol contract. It may be best to not explicitly limit this at a contract level to allow the admin to make important updates in emergency situations.

Furthermore, consider updating the `burnExcessShares` function so that it can successfully execute and early return if the share value has decreased since the withdrawal request was made.

— RESOLUTION

Status: Acknowledged.

LOW **L-19**

RESOLVED

Missing SubConsol Validation

CATEGORY	Validation
LOCATION	GeneralManager.sol
REVIEW	Main Review

DESCRIPTION

In the `requestMortgageCreation` function there is no validation which prevents a user from providing a SubConsol address that is invalid either because it is not supported as a SubConsol address or it does not match the collateral token the user is using.

This leads to orders being created which are inexecutable due to reverts in the `LoanManager.createMortgage` function.

RECOMMENDATION

Add validation in the `requestMortgageCreation` to prevent users from creating orders with a SubConsol address that is not a registered SubConsol contract and does not support the collateral they are providing.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/23>

LOW **L-20**

RESOLVED

Lacking Configs Validation

CATEGORY	Validation
LOCATION	OriginationPoolScheduler.sol
REVIEW	Main Review

DESCRIPTION

In the `addConfig` function there is no validation that prevents a config with a zero consol address from being added, such a config would be able to be added again, creating a duplicate in the `_oPoolConfigIds` list and not allowing this config to be removed.

RECOMMENDATION

Consider validating that the consol on the provided config is not the zero address.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/22>

LOW **L-21**

RESOLVED

Zero Amount Actions Allowed

CATEGORY	Validation
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Across the codebase in the `ForfeitedAssetsPool`, `Consol`, `USDx`, and `SubConsol` contracts 0 value deposits and withdrawals or burns are allowed which may lead to unexpected edge cases or event emissions.

RECOMMENDATION

Validate that the amount being deposited or withdrawn or burned is nonzero to avoid unnecessary paths and unexpected edge cases.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/43>

LOW **L-22**

RESOLVED

Lacking Refund Mechanism

CATEGORY Unexpected Behavior
LOCATION GeneralManager.sol
REVIEW Main Review

DESCRIPTION

In the `GeneralManager.requestMortgageCreation` function among others, there is no ether value refund in the event that the value sent is greater than the order pool's gas fee.

In the event that the default admin re-assigns the gas fee value for the OrderPool before a user's transaction is recorded, this may result in the user sending more gas than is necessary and not receiving an appropriate refund back.

RECOMMENDATION

Consider sending a refund of any excess ether to the caller at the end of the transaction.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/8>

LOW **L-23**

RESOLVED

Lacking SafeCast Usage

CATEGORY —
LOCATION —
REVIEW Main Review

DESCRIPTION

In the `periodsSinceTermOrigination` function the `periods` amount is casted to a `uint8` without checking if the result fits within a `uint8` object. This will result in a perturbation of the resulting periods value when the time passed exceeds 256 periods.

In the `periodsPaid` function the resulting value is casted to a `uint8` without validating that the result fits within this storage.

In the `interestRate` function of the pyth interest rate oracle the rate result is casted to a `uint16` without checking if it fits within this storage.

In the `price` function the price result is casted to a `uint64` value without checking if it fits within this storage.

RECOMMENDATION

Use `safeCast` for all of these casting results as well as all other casting that occurs in the codebase to avoid any unexpected overflows.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/29>

LOW **L-24**

RESOLVED

Convert Rounds In Favor Of The User

CATEGORY Rounding
LOCATION MortgageMath.sol
REVIEW Main Review

— DESCRIPTION

In the `convert` function, when a partial conversion is occurring the remaining `termPaid` is calculated using `Math.Rounding.Ceil`, however the `termPaid` amount should be rounded down to round against the user and in favor of the protocol as a best practice.

— RECOMMENDATION

Round the `termPaid` by the user down instead of up to favor the safety of the protocol.

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/27>

LOW **L-25**

RESOLVED

Lacking Expiration Validation

CATEGORY Validation
LOCATION OrderPool.sol
REVIEW Main Review

DESCRIPTION

The `OrderPool` `sendOrder` function allows orders to be stored that have an expiration that has already passed.

RECOMMENDATION

Do not allow these orders to be stored and instead revert.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/28>

LOW **L-26**

RESOLVED

Ineffective Mortgage Request Fills

CATEGORY Unexpected Behavior
LOCATION Global
REVIEW Main Review

— DESCRIPTION

The fulfiller of a mortgage origination receives no spread on the collateral price for fulfilling the order.

Furthermore, the price used is a static price from when the order was originated, therefore if price increases after the pyth price used at the time of the request initiation then it is unlikely that the order will be fulfilled, and this is simply a waste of the gas downpayment made by the user.

— RECOMMENDATION

Consider improving the fulfillment incentives so that mortgage requests are more likely to be fulfilled.

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/54>

LOW **L-27**

RESOLVED

Actions Can Only Occur During Market Hours

CATEGORY Warning
LOCATION Global
REVIEW Main Review

— DESCRIPTION

Given that the Pyth feed for the US treasury rate is only updated during relevant market hours it is not possible to query the rate with a recent published result during off-market hours.

As a result, actions like loan request creation and refinancing are unavailable during off-hours.

— RECOMMENDATION

Be aware of this behavior and be sure to surface it to users in a palatable way.

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/45>

LOW **L-28**

RESOLVED

Term And Penalty Payments Can Be Grieved

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/libraries/MortgageMath.sol#L281-L283
REVIEW	Main Review

— DESCRIPTION

In the case where a borrower attempts to pay their full penalty or term value, a malicious user can cause the borrower's payment transaction to revert by frontrunning and paying the loan down by 1 wei.

The reason for the revert is because the borrower is not able to overpay their penalties or outstanding principal. The single-wei donation will cause the borrower's full payment to overpay by 1 wei, leading to their payment transaction reverting.

This is possible for all penalty payments and only period payments when the mortgage has a payment plan.

— RECOMMENDATION

The simple solution is to lock down payments only to the mortgage owner. Otherwise, allow the borrower to overpay and simply refund the excess balance at the end of the transaction.

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/17>

LOW **L-29**

RESOLVED

Check Balance In removeAsset

CATEGORY	Unexpected Behavior
LOCATION	—
REVIEW	Main Review

DESCRIPTION

The `removeAsset` function in the `ForfeitedAssetsPool` contract removes the asset from the list without checking the contract balance. If this asset has already been forfeited before, that balance remains locked in the contract unless the asset is added back to the list, since a removed asset cannot be redeemed during a burn.

RECOMMENDATION

Consider checking the contract balance in `removeAsset` and either reverting if the asset has already been forfeited or clearing the balance. However, note that clearing the balance might be unfair to users who are awaiting withdrawal.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/39>

LOW **L-30**

RESOLVED

USDX Withdrawals Can Be DoSed

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/MultiTokenVault.sol#L165-L214
REVIEW	Main Review

DESCRIPTION

The `USDX.sol` contract allows users to deposit and withdraw various USD-valued tokens. Since the deposit and withdrawal flow does not impose fees, users are able to swap between tokens freely.

This poses a problem because when users specify their withdrawal token, a malicious actor can deposit another token and withdraw all of the user's desired token's balance. All withdrawals can be temporarily grieyed through this method.

RECOMMENDATION

Note this possibility in the documentation.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/49>

LOW **L-31**

ACKNOWLEDGED

ForfeitedAssetsQueue Can Be DoSed

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/ForfeitedAssetsQueue.sol#L51
REVIEW	Main Review

— DESCRIPTION

The ForfeitedAssetsQueue handles each withdrawal request sequentially and must have enough liability in the ForfeitedAssetsPool to be able to fulfill the request, otherwise it reverts.

Therefore, a large Consol holder can queue a withdrawal through the ForfeitedAssetsQueue and DoS the queue until there are enough forfeited assets to satisfy the large withdrawal.

The user could cancel their request resulting in simply stalling the distribution of forfeited assets and a delay in allowing users to exit their Consol positions.

This is unlikely to occur in practice because it would take a large financial commitment by the malicious user and provide little to no benefit.

— RECOMMENDATION

Document the possibility of such a scenario.

— RESOLUTION

Status: Acknowledged.

LOW **L-32**

RESOLVED

Incorrect Natspec Comment

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/MortgageQueue.sol#L200
REVIEW	Main Review

DESCRIPTION

The `_findFirstTriggered` function has the following Natspec comment:

"@return tokenId the tokenId of the first MortgagePosition in the Conversion Queue that has a trigger price greater than or equal to the input trigger price."

However, the function returns the position with a trigger price less than or equal to the input trigger price.

RECOMMENDATION

Change the comment.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/38>

LOW **L-33**

RESOLVED

Inactive Mortgages In ConversionQueue

CATEGORY	Unexpected Behavior
LOCATION	ConversionQueue.sol
REVIEW	Main Review

DESCRIPTION

Redeemed or foreclosed mortgages remain in the `ConversionQueue` until the `dequeueMortgage` function is called. If such a mortgage becomes the head of the queue, the `processWithdrawalRequests` function will revert until that mortgage is removed.

RECOMMENDATION

Consider either dequeuing a mortgage when it becomes inactive or skipping it in the while loop of the `processWithdrawalRequests` function.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/48>

LOW **L-34**

RESOLVED

Old forfeitedAssetsPool Remains Supported

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/Consol.sol#L61-L70
REVIEW	Main Review

DESCRIPTION

When updating the `forfeitedAssetsPool` in `Consol.sol`, the previous value of `forfeitedAssetsPool` remains in the `supportedTokens` enumerable set.

The `cap` value for the previous pool is deleted, therefore it would be wise to remove the pool token from the list of supported tokens to avoid its use in the `flashSwap()` function.

RECOMMENDATION

Consider removing the previous `forfeitedAssetsPool` from the enumerable set with `supportedTokens.remove()`.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/18>

LOW **L-35**

RESOLVED

Duplicate Deletion Of Order

CATEGORY	Error
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/OrderPool.sol#L168-L176
REVIEW	Main Review

DESCRIPTION

When an order is expired, the order record is deleted twice. Firstly, in the `if` clause:

```
if (order.expiration < block.timestamp) {  
    // Cancel the mortgage request  
    IGeneralManager(generalManager).burnMortgageNFT(order.mortgageParams.tokenId);  
    // Delete the order  
    delete _orders[index];  
    // Emit the PurchaseOrderExpired event  
    emit PurchaseOrderExpired(index);  
}
```

And finally at the end of function execution:

```
collectedGasFee = order.gasFee;  
// Delete the order  
delete _orders[index];
```

RECOMMENDATION

Remove the deletion in the `if` clause as the order will eventually be deleted at the end of function execution.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/19>

LOW **L-36**

ACKNOWLEDGED

Early Closures Still Pay Full Interest

CATEGORY Informational
LOCATION Global
REVIEW Main Review

— DESCRIPTION

Borrowers may close their mortgages early if they choose. However, closing early does not recalculate the interest or total amount payable, and borrowers are still required to pay interest for the entire mortgage term.

— RECOMMENDATION

Document this behavior and ensure users are aware of it.

— RESOLUTION

Status: Acknowledged.

LOW **L-37**

ACKNOWLEDGED

Race Condition For ForfeitedAssetsPool Tokens

CATEGORY	Logical Error
LOCATION	Consol.sol, MultiTokenVault.sol
REVIEW	Main Review

DESCRIPTION

Consol is a MultiTokenVault that supports USDX and ForfeitedAssetsPool tokens. The total supply of MultiTokenVault s is calculated based on the balances of the supported tokens, and MultiTokenVault assumes that all supported tokens share the same unit of account (UOA).

However, the UOA of the supported tokens in Consol can differ significantly. For example, a single share of ForfeitedAssetsPool can easily be worth 1.50 due to seized collaterals, while USDX is 1.00. As a result, Consol supporting both of these tokens contradicts the intended behavior of a MultiTokenVault .

Since seized assets can be purchased at 1:1 by burning Consol, a race condition can occur when a single share of ForfeitedAssetsPool is worth more than \$1. Users will attempt to withdraw from the ForfeitedAssetsPool first, followed by withdrawals from USDX.

RECOMMENDATION

Consider documenting this if it is the intended behavior. Otherwise, ensure that all supported assets in Consol share the same UOA. This may require calculating the mint amount of ForfeitedAssetsPool tokens at the time of foreclosure based on the value of the seized collateral and the value of Consol at that point.

RESOLUTION

Status: Acknowledged.

LOW **L-38**

RESOLVED

Penalty Calculation Rounds In Favor Of Borrower

CATEGORY	Rounding
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/libraries/MortgageMath.sol#L337-L341
REVIEW	Main Review

DESCRIPTION

When calculating the number of penalties that the borrower owes, the penalty calculation rounds down in favor of the late borrower. Likely this only results in a negligible advantage for the late borrower; however, it is best practice to always round in favor of the protocol.

RECOMMENDATION

Use `Math.Rounding.Ceil` to round in favor of the protocol.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/40>

LOW **L-39**

RESOLVED

Initial oPoolAdmin Does Not Have Admin Role

CATEGORY Configuration

LOCATION <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/0079122e82f9926449b3f736ab4a35ea97f092c0/src/OriginationPoolScheduler.sol#L114>

REVIEW Main Review

DESCRIPTION

`oPoolAdmin` in the `OriginationPoolScheduler` must have the `DEFAULT_ADMIN_ROLE`. However, this role is not granted to `oPoolAdmin` during the initialization process. Instead, it is granted to `msg.sender`, which may not be the `oPoolAdmin`.

RECOMMENDATION

Grant the admin role to `oPoolAdmin` during initialization.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/21>

LOW **L-40**

ACKNOWLEDGED

Penalty Rate Updates Affect Existing Mortgages

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/LoanManager.sol#L75>

REVIEW Main Review

DESCRIPTION

If the admin updates the penalty rate value stored in `GeneralManager.sol`, the currently late mortgages could experience increased penalty fees. These mortgage owners enter into the agreement with a set rate and can experience fluctuations depending on the global penalty rate setting.

RECOMMENDATION

Consider writing the current global `penaltyRate` for the mortgage at the time of origination request and use that value in subsequent penalty calculations.

RESOLUTION

Status: Acknowledged.

LOW **L-41**

RESOLVED

Theft Of Funds From USDX With scalarDenominator

CATEGORY	Rounding
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/MultiTokenVault.sol#L199
REVIEW	Main Review

— DESCRIPTION

There is a possible, but unlikely theft vector within USDX.sol. The USDX contract allows users to deposit and withdraw various USD-valued tokens. Because each token may have different decimals, there is a scaling mechanism that multiplies the token amount by a numerator and divides by a denominator.

The potential theft vector occurs when the denominator is greater than 1, for example 10e12.

Imagine USDX is set up with 6-decimal precision and 18-decimal USD tokens must be scaled down to 6 decimals. When calculating the mint amount of 1e18 token, the resulting amount would be 1e6.

When the user attempted to withdraw 1e18 tokens, the same conversion would take place and require that they burn 1e6 pool tokens.

The theft can occur if the user specifies that they wish to withdraw (1e18 + 1e11) tokens. With precision loss, the amount of pool tokens to burn will still be 1e6.

However, the user will receive 1e11 tokens more than they deposited. They can perform this call many times to drain funds from the pool.

— RECOMMENDATION

Ensure that the denominator is always 1, as is the case in the deployment script.

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/41>

LOW **L-42**

RESOLVED

CEI Pattern Not Followed

CATEGORY	Best Practices
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/LenderQueue.sol#L178-L179
REVIEW	Main Review

— DESCRIPTION

When cancelling a withdraw request, the consol token is transferred to the user, however, the amount is not updated to 0 until after the external call. If Consol were to include a hook for the receiver of the token, the withdrawal queue could be drained. It is unlikely that such would be the case.

— RECOMMENDATION

Move the storage writes above the external call to the consol token.

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/20>

INFORMATIONAL **I-01**

RESOLVED

calculateNewAvergageInterestRate Naming

CATEGORY	Typo
LOCATION	MortgageMath.sol
REVIEW	Main Review

DESCRIPTION

The `calculateNewAvergageInterestRate` function misspells average.

RECOMMENDATION

Correct the spelling of average in the `calculateNewAvergageInterestRate` function name.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/16>

INFORMATIONAL I-02

RESOLVED

Typos

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/LoanManager.sol#L417
REVIEW	Main Review

DESCRIPTION

- Corresponding -> <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/LoanManager.sol#L417>
- Surplus -> <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/LoanManager.sol#L270>
- Additional -> <https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/bc9e1c53bc1f549af634ad100a5c0b6a3f6f54a4/src/interfaces/IGeneralManager/IGeneralManager.sol#L226>

RECOMMENDATION

Consider, fixing these typos.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/42>

HIGH **H-01**

RESOLVED

Some Requests Cannot Be Cancelled

CATEGORY	Logical Error
LOCATION	LenderQueue.sol: 158
REVIEW	Remediation Review 1

— DESCRIPTION

In the `cancelWithdrawal` function the validation on the index ensures that the index is less than the `withdrawalQueueLength`. However, the `withdrawalQueueLength` is variable and the indexes are now fixed.

Therefore, there can be a withdrawal at absolute index 10, while the withdrawal head is at 10 and the withdrawal queue length is 1. The index provided to the `cancelWithdrawal` function must be 10, which is greater than the `withdrawalQueueLength` of 1.

Consider the following example:

- The withdrawal queue is empty with the head being 0 and length being 0
- Bob creates withdrawal A, which is stored at index 0, the head is still 0 and length is now 1
- Alice creates withdrawal B, which is stored at index 1, the head is still 0 and length is now 2
- Bob's withdrawal A is now processed; the head is at 1 and the length is now 1
- Alice attempts to cancel her withdrawal, however providing the index 1 fails the index validation and she cannot cancel

Furthermore, the current validation allows requests that are before the head to be cancelled, however these requests have already been processed.

— RECOMMENDATION

Correct the index validation such that only indexes that satisfy `head <= index < head + length` are allowed.

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/57>

HIGH **H-02**

RESOLVED

amountForfeited Credits Mortgage Holders

CATEGORY Logical Error
LOCATION MortgageMath.sol
REVIEW Remediation Review 1

— DESCRIPTION

The amountConverted variable has been changed to represent the amount of debt converted in past terms. And the current term converted value is now tracked in the termConverted variable. This subtly changes all formulas that currently use amountConverted, and if termConverted is not also tracked there this causes notable accounting issues.

The amountForfeited function uses amountConverted but does not account for the newly introduced termConverted correctly.

The original implementation of amountForfeited was as follows:

$$\text{amountBorrowed} - \text{amountConverted} - \text{amountOutstanding}$$

Where $\text{amountOutstanding} = \text{amountBorrowed} - \text{amountConverted} - \text{amountPrior} - \text{convertToPrincipal}(\text{termPaid})$

And in this context amountConverted represents the entire amount converted across all terms for the mortgage.

This simplifies to:

$$\text{amountBorrowed} - \text{amountConverted} - \text{amountOutstanding}$$
$$= \text{amountBorrowed} - \text{amountConverted} - (\text{amountBorrowed} - \text{amountConverted} - \text{amountPrior} - \text{convertToPrincipal}(\text{termPaid}))$$
$$= \text{amountPrior} + \text{convertToPrincipal}(\text{termPaid})$$

This is correct because this is the amount of debt that the user has already paid and is forfeiting during liquidation.

However, the new implementation of amountForfeited is as follows:

$$\text{amountBorrowed} - \text{amountConverted} - \text{principalRemaining}$$

Where $\text{principalRemaining} = \text{amountBorrowed} - \text{amountConverted} - \text{amountPrior} - \text{convertToPrincipal}(\text{termPaid} + \text{termConverted})$

H-02 amountForfeited Credits Mortgage Holders

CONTINUED

And in this context amountConverted represents the entire amount converted across only past terms for the mortgage.

This simplifies to:

$$\text{amountBorrowed} - \text{amountConverted} - \text{principalRemaining}$$
$$= \text{amountBorrowed} - \text{amountConverted} - (\text{amountBorrowed} - \text{amountConverted} - \text{amountPrior} - \text{convertToPrincipal}(\text{termPaid} + \text{termConverted}))$$
$$= \text{amountPrior} + \text{convertToPrincipal}(\text{termPaid} + \text{termConverted})$$

This is incorrect because now the mortgage holder is credited with forfeiting the converted amount in the current term.

— RECOMMENDATION

Update the forfeitedAssets function as follows:

```
function amountForfeited(MortgagePosition memory mortgagePosition) internal pure returns
(uint256) {
    if (mortgagePosition.status != MortgageStatus.FORECLOSED) {
        return 0;
    }
    - return mortgagePosition.amountBorrowed - mortgagePosition.amountConverted -
mortgagePosition.principalRemaining();
    + return mortgagePosition.amountPrior +
mortgagePosition.convertPaymentToPrincipal(mortgagePosition.termPaid);
}
```

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/72>

HIGH **H-03**

RESOLVED

Gas Fees Trapped In enqueueMortgage

CATEGORY	Logical Error
LOCATION	GeneralManager.sol
REVIEW	Remediation Review 1

DESCRIPTION

The `enqueueMortgage` function uses the `_calculateRequiredGasFee` function to calculate the `msg.value` that is required to send to all of the conversion queues being added with the `conversionQueueList` parameter.

However, the `_calculateRequiredGasFee` function computes the required gas fees for all conversion queues that the `tokenId` currently has in their `conversionQueues(tokenId)` list, not just the ones that are being added with the call to `enqueueMortgage`.

Therefore, the `requiredGasFee` is larger than what is necessary to send to the new conversion queues, and this amount is lost to the mortgage owner.

RECOMMENDATION

Instead of using the `_calculateRequiredGasFee` function which computes the necessary gas fee for all of the conversion queues, old and new. Implement a bespoke for-loop to loop through all of the new conversion queues in the provided `conversionQueueList` parameter.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/73>

HIGH **H-04**

RESOLVED

Missed Payments Perturbed

CATEGORY Logical Error
LOCATION MortgageMath.sol
REVIEW Remediation Review 1

DESCRIPTION

In the `applyPenalties` function, for mortgages that do not have a payment plan, the `additionalPaymentsMissed` are assigned to:

```
periodsSinceOrigination - mortgagePosition.totalPeriods + 1 - mortgagePosition.paymentsMissed
```

However, in the `periodPay` and `convert` functions, the `mortgagePosition.paymentsMissed` value is assigned to the following for mortgages with and without payment plans:

```
mortgagePosition.paymentsMissed = _periodsPaid > periodsSinceOrigination ? 0 :  
periodsSinceOrigination - _periodsPaid;
```

There are several issues with this, the most glaring being that in conversion the `paymentsMissed` update does not account for the fact that mortgages without a payment plan cannot have missed payments before their mortgage is complete. This is not an issue in the `periodPay` function since mortgages without payment plans must be totally paid off when invoking the `periodPay` function.

Secondly, if the `periodsSinceOrigination` is larger than the total mortgage periods then the `paymentsMissed` will always be assigned to a nonzero amount even if the total debt has been paid off. The `paymentsMissed` logic should be updated like so:

```
uint8 periodsSinceOrigination =  
mortgagePosition.periodsSinceTermOrigination(latePenaltyWindow);  
+ uint8 cappedPeriodsPassed = Math.min(periodsSinceOrigination, mortgage.totalPeriods);  
uint8 _periodsPaid = mortgagePosition.periodsPaid();  
- mortgagePosition.paymentsMissed = _periodsPaid > periodsSinceOrigination ? 0 :  
periodsSinceOrigination - _periodsPaid;  
+ mortgagePosition.paymentsMissed = _periodsPaid > cappedPeriodsPassed ? 0 :  
cappedPeriodsPassed - _periodsPaid;
```

H-04 Missed Payments Perturbed

CONTINUED

Thirdly, both the `periodPay` function and the `convert` function do not account for the `+ 1` that is applied to the `additionalPaymentsMissed` in the `applyPenalties` function and therefore overwrite this additional missed payment period for mortgages that do not have a payment plan.

— RECOMMENDATION

Firstly, in the `convert` function, be sure to account for the fact that mortgages without a payment plan cannot have missed payments before their term is up.

Secondly, account for the additional payment period that is applied to non-payment-plan mortgages in the `periodPay` and `convert` functions.

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/74>

HIGH **H-05**

RESOLVED

Rounding Prevents Action Cancellations

CATEGORY Rounding
LOCATION Global
REVIEW Remediation Review 1

DESCRIPTION

In the `cancelWithdrawal` function in some cases the number of shares being transferred for the `request.amount` will round to be one wei greater than the number of shares that the `LenderQueue` holds for that withdrawal request after the `burnExcessShares` invocation.

In cases where there is only one withdrawal request in the `LenderQueue`, the cancellation will revert due to `ERC20InsufficientBalance`, and in the case where there are multiple withdrawal requests, the cancellation will take 1 wei that should have been allocated to other withdrawal requests, ultimately causing a revert for the last withdrawal being cancelled or even processed.

RECOMMENDATION

The rounding of `burnExcessShares` and the subsequent transfer must be overhauled to avoid sending an additional wei. A simple solution may be to transfer `request.amount - 1` to ensure this case does not arise.

PROOF OF CONCEPT <https://github.com/GuardianOrg/cash-buttonwood-cash-fuzz/blob/81a67f4b54cd8b1a18342ef10e1584328680654d/test/fuzzing/FoundryPlayground.sol#L85>

RESOLUTION

Status: Resolved.

HIGH **H-06**

RESOLVED

Multiple Origination Pools Broken

CATEGORY	Logical Error
LOCATION	GeneralManager.sol
REVIEW	Remediation Review 1

DESCRIPTION

The latest logic in the origination flow sends the available consol token balance of the GeneralManager contract to the immediate Origination pool being processed.

```
IERC20($_.consol).safeTransfer(_msgSender(), IConsol($_.consol).balanceOf(address(this)));
```

This however leaves an insufficient amount of consol tokens to repay the following origination pools. This is masked by the following logic:

```
uint256 consolBalance = IConsol($_.consol).balanceOf(address(this));  
if (consolBalance < returnAmount) {  
    IConsol($_.consol).deposit($_.usdx, IConsol($_.consol).convertUnderlying($_.usdx, returnAmount  
    - consolBalance));  
}
```

Which will take from the USDX balance of the contract to repay the following origination pools. This is not surfaced immediately as an error because the fulfiller now receives the balance of the contract instead of the purchaseAmount they should have received.

RECOMMENDATION

Instead of transferring the entire GeneralManager balance to the immediate origination pool being processed, transfer the specified returnAmount.

This way there still may be small rounding which affects the fulfiller, but it is not confused with an amount that covers a dearth for other origination pools.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/76>

MEDIUM **M-01**

RESOLVED

Mortgage Fee Lost On Expiration

CATEGORY Warning
LOCATION OrderPool.sol
REVIEW Remediation Review 1

DESCRIPTION

In the `OrderPool` contract the executor of the `processOrders` function has an adverse incentive to wait until orders are expired before processing them, this is because the `order.mortgageGasFee` is added to the `collectedGasFee` in `_processOrder` when an order has expired.

This means that when an order expires the user loses their `mortgageGasFee` even though a mortgage was not created, and the executor gets to keep it for themselves.

RECOMMENDATION

If an order expires, still collect the `order.orderPoolGasFee`, however the `order.mortgageGasFee` should be refunded to the user. It should be refunded by incrementing a mapping value entry and allowing the user to claim this Ether in a separate transaction to avoid any DoS vectors or gas griefing vectors. Or by wrapping the Ether to wrapped Ether which does not allow for these attacks.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/69>

MEDIUM **M-02**

ACKNOWLEDGED

Risk Free Time Difference Arbitrage

CATEGORY Gaming
LOCATION Global
REVIEW Remediation Review 1

— DESCRIPTION

When a mortgage creation request is created, the purchaseAmount and collateralAmount are defined based on the price of the asset using the pyth oracle in the block of the creation request.

The creation request is then executed up to 5 minutes later, using these prices from as old as 5 minutes ago.

This allows users to potentially arbitrage price movements which occur in those 5 minute timeframe, by deciding whether or not their order is executed.

For example:

- At time 100, User A makes their expand balance sheet request when Bitcoin is \$100,000
- User A configures their creation request to enqueue into a conversion queue that they are already enqueued in
- User A sets their receive function to revert based on a boolean switch, which is currently set to true to prevent the order from being executed
- At time 110, Bitcoin rises to \$110,000
- User A sees that Bitcoin price has moved in their favor and flips the boolean switch allowing the receive function to no longer revert and the refund that occurs on the re-enqueue action to proceed
- User A's action is now processed using an outdated price of \$100,000 per Bitcoin for their borrow
- If price had not moved in User A's favor, then they would have just left the switch and allowed their order to be cancelled.

— RECOMMENDATION

Be aware of this time difference arbitrage risk, the OrderPool execution fee may be enough to disincentivize this, furthermore the net interest rate paid on a mortgage will always overshadow what can be arbitrated through this method in the current expiration window.

M-02 Risk Free Time Difference Arbitrage

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Collateral Consumed At Stale Trigger Price

CATEGORY Logical Error
LOCATION ConversionQueue.sol: 117
REVIEW Remediation Review 1

— DESCRIPTION

When processing withdrawals, `collateralToUse` is computed by dividing the payment by the stored `triggerPrice` rather than the current oracle price. If the market price has risen above the trigger price, the system will consume more collateral than necessary for the same repayment.

While the borrower accepted the trigger price for conversion, this should not mean that the borrower converts their collateral at a rate lower than its market value. Additionally, the comment in the code states: 'Figure out how much collateral corresponds to the amountToUse at the current price.'

— RECOMMENDATION

Consider using the current value of the collateral during conversions, as in the previous version. However, if this change is an intentional design choice, clearly document this behavior for users, since they will convert their collateral at a lower rate unless the trigger price matches the current value. In that case, also update the comment accordingly.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

RESOLVED

Converted No Payment Plan Mortgages

CATEGORY Logical Error
LOCATION MortgageMath.sol
REVIEW Remediation Review 1

DESCRIPTION

In the `periodPay` function validation is performed on mortgages with no payment plan to ensure that the amount is at least the `mortgagePosition.termBalance`.

However, for mortgages with no payment plan that have been partially converted, there isn't enough debt to pay off the entire `termBalance`, since the `termConverted` is reducing a portion of this debt.

Due to the refund logic this doesn't strictly prevent these mortgages from being paid off, however users must obtain and approve more than their actual debt amount of Consol to pay off their remaining debt in this case.

RECOMMENDATION

Update the validation to:

```
if (!mortgagePosition.hasPaymentPlan && amount < mortgagePosition.termRemaining()) {  
  revert CannotPartialPrepay(mortgagePosition);  
}
```

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/77>

MEDIUM **M-05**

ACKNOWLEDGED

No Payment Plan Mortgages Arbitraged

CATEGORY Gaming
LOCATION Global
REVIEW Remediation Review 1

— DESCRIPTION

Mortgages without payment plans require that their entire debt is paid off in a single `periodPay` invocation.

However, these mortgages in particular, create a large arbitrage opportunity when they pay back their debt. Where the forfeited Consol that pays the interest on the loan is forfeited, increasing the Consol price.

This may be abused by third parties by depositing into USDx, then obtaining Consol tokens right before the large payment and then immediately exiting one of the available queues, likely the USDx queue using their own deposited USDx and the mortgage holders USDx, as long as this queue is not too full.

Furthermore, this may serve as an avenue for the mortgage holder to avoid paying interest. If they are able to flash loan or make a short period loan of a large amount of Consol tokens, they themselves could absorb much of the Consol price increase and re-coup their interest payment.

— RECOMMENDATION

Be aware of this heightened risk of gaming, particularly for mortgages without a payment plan.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Empty Requests Can Be Cancelled

CATEGORY Validation
LOCATION LenderQueue.sol: 156
REVIEW Remediation Review 1

DESCRIPTION

In the `cancelWithdrawal` function there is no validation that prevents an empty, already cancelled, withdrawal from being processed again.

There is no net effect of processing such a withdrawal, however this execution path should be limited to avoid unexpected outcomes and the emission of a misleading `WithdrawalCancelled` event.

RECOMMENDATION

Validate that either of the request shares and amount must be nonzero to allow processing to occur.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/68>

LOW **L-02**

ACKNOWLEDGED

safeTransferFrom Should Come First

CATEGORY Best Practices
LOCATION MultiTokenVault.sol
REVIEW Remediation Review 1

— DESCRIPTION

In the `MultiTokenVault` `deposit` function the `safeTransferFrom` action comes after the minting of tokens to the user, therefore the user receives shares before officially paying for them.

To avoid any unexpected re-entrancies with arbitrary supported tokens, the `safeTransferFrom` should occur before the `_mint` action so there is no point where the protocol is in an invalid state at the time of an external call.

— RECOMMENDATION

Perform the `safeTransferFrom` action before the mint action in the `deposit` function.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

More Than 18 Decimals Incompatible

CATEGORY Validation
LOCATION USDX.sol: 72
REVIEW Remediation Review 1

DESCRIPTION

In the `addSupportedToken` function validation has been added to ensure that the `scalarNumerator` is greater than or equal to the `scalarDenominator`.

This disallows tokens with greater than 18 decimals from being used with the system, such as yamV2 which has 24 decimals.

RECOMMENDATION

Be aware of this incompatibility or consider removing the validation.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

Lacking CEI In Burn

CATEGORY Best Practices
LOCATION MultiTokenVault.sol
REVIEW Remediation Review 1

DESCRIPTION

In the USDX burn function the USDX burn occurs for the user at the end of the function after all tokens have been redeemed to the user. However, to follow CEI the burn should occur first.

RECOMMENDATION

Invoke `_burn` after initial validations in the burn function.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/59>

LOW **L-05**

ACKNOWLEDGED

Dangerous Withdrawal Request Amounts

CATEGORY	Validation
LOCATION	LenderQueue.sol
REVIEW	Remediation Review 1

— DESCRIPTION

In the requestWithdrawal function if the amount is a few wei the action may produce a withdrawalRequest that holds a nonzero amount value but a zero shares value.

This creates an unexpected case in the queues that attempt to process such withdrawals.

Furthermore, if the amount specified is zero, this allows users to create withdrawals with both zero-amount value and zero share value.

— RECOMMENDATION

Both of these cases are disallowed when the minimumWithdrawalAmount is assigned to a non-trivial value. Ensure that the minimumWithdrawalAmount is always assigned to a minimum reasonable amount to ensure no game-ability and rounding issues are present in all queues.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

RESOLVED

Lacking Pausability Guards

CATEGORY	Validation
LOCATION	Global
REVIEW	Remediation Review 1

DESCRIPTION

The `ConversionQueue` has a `whenNotPaused` guard which applies to the `processWithdrawalRequests` function, however the `USDQueue` and `ForfeitedAssetsQueue` do not implement such a validation.

RECOMMENDATION

Consider if the `USDQueue` and `ForfeitedAssetsQueue` should use a `whenNotPaused` modifier for their `processWithdrawalRequests` functions.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/67>

LOW **L-07**

RESOLVED

Missing Reentrancy Guard

CATEGORY	Validation
LOCATION	OrderPool.sol
REVIEW	Remediation Review 1

DESCRIPTION

In the `OrderPool` the `processOrders` function is the only function which has a `nonReentrant` modifier which provides limited protection against reentrancy into the `processOrders` function.

The `sendOrder` function still poses a risk of unexpected reentrancy and therefore should also use the `nonReentrant` modifier so that it cannot be unexpectedly entered into during a `processOrder` execution.

RECOMMENDATION

Add a `nonReentrant` guard to the `sendOrder` function.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/61>

LOW **L-08**

ACKNOWLEDGED

Blacklist Prevents Order Processing

CATEGORY	Warning
LOCATION	OrderPool.sol
REVIEW	Remediation Review 1

— DESCRIPTION

In the case that an order is expired, the collected collateral and USDX are returned to the user and the executor gets to collect the associated gas fee.

However, if the user happens to be blacklisted for the collateral token, the `_processOrder` function will not be able to execute and process their order as it attempts to send tokens to a blacklisted address.

— RECOMMENDATION

Be aware of this in the event that any supported collateral tokens implement a blacklist. Consider using a mapping to track balances that a user may claim in a separate transaction rather than pushing assets to the user.

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

RESOLVED

Lacking Processing CEI

CATEGORY Warning
LOCATION OrderPool.sol
REVIEW Remediation Review 1

DESCRIPTION

In the `_processOrder` function the `_orders` mapping is cleared at the end of the function, however, to follow CEI it would be best if this mapping entry is cleared at the beginning of the function.

RECOMMENDATION

Clear the `_orders` mapping at the beginning of the `_processOrder` function directly after caching the order in memory.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/62>

LOW **L-10**

RESOLVED

Lacking Length Validation

CATEGORY	Validation
LOCATION	GeneralManager.sol: 867
REVIEW	Remediation Review 1

DESCRIPTION

When creating a mortgage request the `collateralAmounts` length must match the `originationPools` length, however this is not validated in the `requestMortgageCreation` function.

RECOMMENDATION

Add validation so that the `collateralAmounts` length is required to match the `originationPools` length.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/66>

LOW **L-11**

ACKNOWLEDGED

Outdated Gas Fee Validation

CATEGORY Validation
LOCATION OrderPool.sol: 133
REVIEW Remediation Review 1

DESCRIPTION

In the `OrderPool` contract `sendOrder` function the validation performed on the `msg.value` is still against the `gasFee`, however the `gasFee` only represents the order pool gas fee and does not include the mortgage gas fee that should be provided.

RECOMMENDATION

Update the `msg.value` validation in the `sendOrder` function so that it includes the `_calculateMortgageGasFee` result.

RESOLUTION

Status: Acknowledged.

LOW **L-12**

RESOLVED

Typo

CATEGORY	Typo
LOCATION	GeneralManager.sol
REVIEW	Remediation Review 1

DESCRIPTION

A comment in the `originationPoolDeployCallback` function states:

```
// Send in the collateral to the LoanManager before creating the origination pool
```

However, this should read:

```
// Send in the collateral to the LoanManager before creating the mortgage
```

RECOMMENDATION

Correct the comment.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/63>

LOW **L-13**

RESOLVED

Dangerous Conversion Queue Refund

CATEGORY	Best Practices
LOCATION	ConversionQueue.sol
REVIEW	Remediation Review 1

— DESCRIPTION

When enqueueing a mortgage in the `ConversionQueue`, if the mortgage was previously enqueued it is first removed from the queue to be re-enqueued. In this process the original gas fee is returned to the user, however this allows the user to execute arbitrary action in their receive function.

This may cause unexpected issues due to the user being able to DoS, gas grief the executor, or re-enter into the system unexpectedly.

— RECOMMENDATION

Instead of sending Ether directly to the user, consider incrementing a mapping value where they can claim their owed Ether in a separate transaction. Otherwise wrap the Ether and send it to the user as to avoid triggering untrusted code.

— RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/69>

LOW **L-14**

ACKNOWLEDGED

Lacking Individual Borrow Validation

CATEGORY	Validation
LOCATION	GeneralManager.sol
REVIEW	Remediation Review 1

— DESCRIPTION

In the `_validateBorrowCaps` function there are some baseline validations that the borrowed amounts are within a set range. However, there is no validation that the individual borrows amounts in each `originationParameters.borrowAmounts` list entry are within an expected range.

Therefore, while the total may be inside the expected range, the amount requested from each origination pool may be unexpected.

— RECOMMENDATION

Consider validating that each origination pool requested amount is above some minimum expected amount, or at least above 0.

— RESOLUTION

Status: Acknowledged.

LOW **L-15**

RESOLVED

Duplicate Origination Pools

CATEGORY	Validation
LOCATION	GeneralManager.sol
REVIEW	Remediation Review 1

DESCRIPTION

Now multiple origination pools are allowed to be used for an origination of a mortgage. However, there is no validation to ensure that each entry of the `originationParameters.originationPools` list is unique upon creation of the request.

RECOMMENDATION

Consider validating that each entry is unique when a mortgage request is being created.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/71>

LOW **L-16**

RESOLVED

Lacking Reentrancy Guards

CATEGORY	Validation
LOCATION	GeneralManager.sol
REVIEW	Remediation Review 1

DESCRIPTION

Throughout the `GeneralManager` there are important functions which could be reentered into which are lacking reentrancy guards.

These functions include:

- `requestMortgageCreation`
- `requestBalanceSheetExpansion`
- `originate`
- `enqueueMortgage`
- `convert`

RECOMMENDATION

Add the `nonReentrant` modifier to these functions.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/65>

LOW **L-17**

RESOLVED

Unused subConsol Variable

CATEGORY	Superfluous Code
LOCATION	ConversionQueue.sol: 44
REVIEW	Remediation Review 1

DESCRIPTION

In the `ConversionQueue` contract the `subConsol` storage variable is unused.

RECOMMENDATION

Consider removing it.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/64>

LOW **L-18**

RESOLVED

Incorrect NatSpec Comment

CATEGORY Informational
LOCATION QueueProcessor.sol: 11
REVIEW Remediation Review 1

DESCRIPTION

```
/**  
 * @title QueueProcessor  
 * @author SocksNFlops  
 * @notice The StaticInterestRateOracle contract is a contract that returns a static interest  
rate for new Mortgages being originated. //@audit-issue L incorrect Natspec comment  
 */
```

The @notice comment in QueueProcessor contract belongs to StaticInterestRateOracle.

RECOMMENDATION

Update the comment.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/57>

LOW **L-19**

ACKNOWLEDGED

Redundant Check In convert

CATEGORY	Superfluous Code
LOCATION	MortgageMath.sol: 673
REVIEW	Remediation Review 1

DESCRIPTION

The `convert` function in the MortgageMath library checks that the current price is greater than the `conversionTriggerPrice`.

```
if (mortgagePosition.conversionTriggerPrice() > currentPrice) {  
    revert ConversionTriggerPriceNotMet(mortgagePosition, currentPrice);  
}
```

However, the same check is performed again at line [673](https://github.com/GuardianOrg/cash-buttonwood-cash-team2/blob/5e435899a100502ebe4ec3bfb42dd8b0f91a2862/src/libraries/MortgageMath.sol#L673), making it redundant.

RECOMMENDATION

Remove the redundant check.

RESOLUTION

Status: Acknowledged.

LOW **L-20**

ACKNOWLEDGED

Redundant Values

CATEGORY	Logical Error
LOCATION	GeneralManager.sol: 88-89
REVIEW	Remediation Review 1

DESCRIPTION

When a mortgage is created with a conversion queue or enqueued later, the `_conversionQueues` and `_mortgageEnqueued` mappings are populated. However, these mappings are never cleared, even though the mortgage is eventually fully converted.

A fully converted mortgage is removed from its respective `ConversionQueue` during the `_popMortgage` call. However, it still remains in other `ConversionQueue`s and must be popped from these queues during withdrawal request processing. Additionally, it remains in the `GeneralManager` mappings as if it were still in the queue.

RECOMMENDATION

Consider updating or clearing the `_conversionQueues` and `_mortgageEnqueued` mappings in `GeneralManager` when a mortgage is fully converted. However, care must be taken to ensure that this does not cause a DoS when the mortgage still needs to be popped from other `ConversionQueue`s during loop iterations.

RESOLUTION

Status: Acknowledged.

LOW **L-21**

ACKNOWLEDGED

Forfeited Assets Queue Held Up

CATEGORY Warning
LOCATION ForfeitedAssetsQueue.sol
REVIEW Remediation Review 1

DESCRIPTION

The `ForfeitedAssetsQueue` can be held up by large withdrawal requests which cannot be fully processed because the `Consol` holdings have not acquired enough `ForfeitedAssetsPool` tokens to withdraw.

This can reasonably occur especially for foreclosures where the remaining debt of the position is small at the time of foreclosure, in which case it would require many such foreclosures to occur to allow the forfeited assets queue to fill a significant withdrawal in the queue.

RECOMMENDATION

In future iterations consider allowing partial withdrawals. Otherwise, be aware of this blocking behavior and communicate it with users.

RESOLUTION

Status: Acknowledged.

LOW **L-22**

ACKNOWLEDGED

Missing requestWithdrawal Refund

CATEGORY Warning
LOCATION LenderQueue.sol
REVIEW Remediation Review 1

DESCRIPTION

The `requestWithdrawal` function does not implement a refund mechanism of too much ether is sent, this does not match other functions in the `GeneralManager` that do perform refunds for native ETH.

RECOMMENDATION

Consider introducing a refund for the `requestWithdrawal` function in future iterations.

RESOLUTION

Status: Acknowledged.

LOW **L-23**

RESOLVED

Incorrect Event Emission

CATEGORY	Logical Error
LOCATION	USDX.sol: 171
REVIEW	Remediation Review 1

DESCRIPTION

In the `burn` function the Withdraw event for the final withdrawal is intended to have the remaining amount - totalBurned, however this case is entered for all withdrawals that come before the final withdrawal, because the main if case is `i == supportedTokens.length() - 1`, meaning that the last withdrawal goes into the first if branch.

RECOMMENDATION

Update the if case to use the condition `i != supportedTokens.length() - 1`.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/77>

LOW **L-24**

RESOLVED

Count Unexpectedly Incremented Twice

CATEGORY	Unexpected Behavior
LOCATION	ConversionQueue.sol
REVIEW	Remediation Review 1

DESCRIPTION

In the `ConversionQueue`, the count variable which is meant to track iterations of the processing loop can unexpectedly be incremented twice in a single loop iteration when the `amountToUse` matches both the `request.amount` and the `mortgagePosition.principalRemaining()`.

This may be unexpected for the caller of the `processWithdrawalRequests` and may result in less withdrawal requests being processed than expected.

RECOMMENDATION

Be aware of this edge case and inform callers of the `processWithdrawalRequests` function.

RESOLUTION

Status: Resolved.

<https://github.com/buttonwood-protocol/cash/pull/77>

HIGH **H-01**

ACKNOWLEDGED

Multiple Origination Pools Broken

CATEGORY	Logical Error
LOCATION	GeneralManager.sol
REVIEW	Remediation Review 2

DESCRIPTION

The latest logic in the origination flow sends the available consol token balance of the GeneralManager contract to the immediate Origination pool being processed.

```
IERC20($_consol).safeTransfer(_msgSender(), IConsol($_consol).balanceOf(address(this)));
```

This however leaves an insufficient amount of consol tokens to repay the following origination pools. This is masked by the following logic:

```
uint256 consolBalance = IConsol($_consol).balanceOf(address(this));
if (consolBalance < returnAmount) {
    IConsol($_consol).deposit($_usdx, IConsol($_consol).convertUnderlying($_usdx, returnAmount
    - consolBalance));
}
```

Which will take from the USDX balance of the contract to repay the following origination pools. This is not surfaced immediately as an error because the fulfiller now receives the balance of the contract instead of the purchaseAmount they should have received.

RECOMMENDATION

Instead of transferring the entire GeneralManager balance to the immediate origination pool being processed, transfer the specified returnAmount.

This way there still may be small rounding which affects the fulfiller, but it is not confused with an amount that covers a dearth for other origination pools.

RESOLUTION

Status: Acknowledged.

<https://github.com/buttonwood-protocol/cash/pull/76>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.